

Efficient Privacy-Preserving Geographic Keyword Boolean Range Query Over Encrypted Spatial Data

Zhimao Gong¹, Junyi Li¹, Yaping Lin¹, *Member, IEEE*, Jianhao Wei², and Camara Lancine

Abstract—With the widespread popularity of mobile devices and geolocation-related services, spatial keyword data has exploded in recent years. As an application, people are accustomed to using specific keywords to search for data in a given geometric range. To protect user privacy, searchable encryption technologies are used to encrypt data and user queries. Most existing works focus on either spatial attributes or keyword attributes over encrypted spatial keyword data, which cannot solve the problem of geographic keyword range queries directly. And several other works considering these two attributes have some limitations in terms of query efficiency and security assurance. In this article, we propose an efficient privacy-preserving geographic keyword Boolean range query (EPBRQ) scheme to solve existing challenges in the current work. In particular, we design a recoding algorithm to break the limits of the current work to achieve lower time complexity and employ secure Knn computation to protect user data privacy comprehensively. The security analysis shows that our solution can well protect the privacy of data and queries from cloud server threats. And numerous experiments based on real-world data also show that our scheme provides better query efficiency than existing works.

Index Terms—Geographic keyword range queries, privacy-preserving, searchable encryption.

I. INTRODUCTION

SPATIAL data have gained an explosive growth due to the wide use of location-based services [1], in which people are used to searching data with special keywords within a given geometric range. For instance, nowadays, a newcomer, when he/she comes to a city, is used to query the records with keyword “COVID-19” for the given areas for safety. Collectively, we refer to this kind of service as geographic keyword Boolean range query (BRQ) [2]. In BRQ, as illustrated in Fig. 1, the

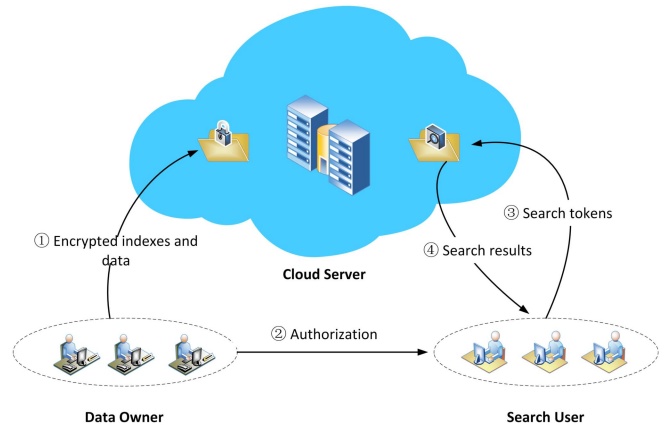


Fig. 1. System model.

data owner (DO) uploads the spatial data including locations and keywords to the cloud server. The search user (SU) issues the Boolean queries with a geometric range and keywords to the cloud. Afterward, the cloud server carries out the search and returns the matched data to the search user. Due to the potential threat of inside attackers and the dishonest cloud server, however, BRQ services are faced with the privacy disclosure challenge [3], especially in sensitive location-based services.

A common way to protect privacy against dishonest cloud server and attackers is to encrypt the spatial data before uploading. In this way, the cloud server cannot learn any true information from the ciphertext without the secret key. However, to provide search services, the server needs to know whether the encrypted data matches a search request. Searchable encryption (SE) [4] is a potentially powerful technology to solve the challenge, allowing the cloud server to perform search functionalities over encrypted data with special trapdoors. Under the constraints of SE, the cloud server can correctly perform the preagreed operations without knowing any real information.

To the best of our knowledge, SE research on textual keyword has been extensive. Many frameworks have been proposed for different functional scenarios, such as multikeywords queries [5], top-k queries [6], [7], and Boolean queries [8], [9]. As for SE research on geometric range query, Wang *et al.* [10]–[12] proposed several schemes using Shen-Shi-Waters (SSW) [13] encryption to implement secure geometric range queries for arbitrary search shapes. Unfortunately, all these schemes are not applicable to large-scale data. Xu *et al.* [14] fitted curve functions for the search range with the help of

Manuscript received 3 September 2021; revised 24 January 2022 and 5 May 2022; accepted 8 June 2022. Date of publication 29 June 2022; date of current version 24 February 2023. This work was supported in part by the National Natural Science Foundation of China under Grant 61872131 and in part by the Special Funds for Construction of Innovative Provinces in Hunan Province of China under Grant 2020GK2006 and Grant 2020GK2007. (Corresponding author: Junyi Li.)

Zhimao Gong, Junyi Li, and Yaping Lin are with the College of Computer Science and Electronic Engineering, Hunan University, Changsha 410082, China, and also with the Hunan Provincial Key Laboratory of Blockchain Infrastructure and Application, Hunan University, Changsha 410012, China (e-mail: javadazhancijia@hnu.edu.cn; junyilee@hnu.edu.cn; yplin@hnu.edu.cn).

Jianhao Wei is with the School of Computer Science, Hunan University of Technology and Business, Changsha 410205, China (e-mail: jianhao@hnu.edu.cn).

Camara Lancine is with the Social Sciences and Management, University of Bamako, Bamako 2735, Mali (e-mail: lcamaralancine@yahoo.fr).

Digital Object Identifier 10.1109/JSYST.2022.3183153

polynomial fitting technique and performed geometric range queries by comparing the function values of data coordinates, while employing secure Knn calculation [15] to protect the privacy. However, their scheme sacrifices query accuracy. Furthermore, the above schemes consider geographic information privacy and textual keyword privacy separately, and cannot be directly applied to spatial keyword queries. Simply combining the textual keyword privacy-preserving scheme and the geographic information privacy-preserving scheme seems to be a straightforward method. However, it produces a lot of useless intermediate results, leading to heavy key management.

To support spatial keyword queries, some hybrid index structures are proposed. Zhou *et al.* [16] combined inverted files and R*-trees into a hybrid index structure that supports location-based web search for the first time. And De Felipe *et al.* [17] provided another hybrid index IR*-tree, a combination of an R-tree and signature files, to support top-*k* spatial keyword queries. Further, Zhang *et al.* [18] designed an IL-Quadtree to achieve efficient top-*k* spatial keyword queries. However, none of these hybrid structures consider privacy issues. To support secure spatial keyword queries, recently, Wang *et al.* [19] designed a BQ-tree encrypted with symmetric-key hidden vector encryption (SHVE) [20], a searchable encryption method that supports conjunctive on encrypted data. However, the employed encryption method SHVE poses the following two problems. 1) *Computational efficiency limitation*: SHVE only supports comparisons between points and ranges. Each node requires five point-to-range comparisons to determine whether two ranges intersect, which leads to high computational complexity. 2) *Trapdoor index leakage*: SHVE leaks some vector subscripts when generating trapdoors. An attacker can recover the trapdoor index by these subscripts and deduce the textual keywords under strong background knowledge. Based on these conditions, there is still a demand to design an efficient and secure solution for geographic keyword range queries.

In this article, we focus on the geographic keyword Boolean range queries, which enable the user to perform geometric range queries with spatial keywords securely and efficiently over encrypted spatial data. To achieve this goal, the following challenges should be solved.

- 1) *Computational efficiency*: Intersection determination of geometric ranges is a very frequent operation during the query process. However, the current work has to complicate such comparison due to the characteristics of the adopted encryption technology. Therefore, designing a method that supports fast geometric range intersection judgment is a significant improvement, but also a challenge.
- 2) *Spatial and textual keyword privacy*: The privacy security of data and queries in spatial or textual keyword privacy-preserving schemes has been proven to be trustworthy. However, when the work involves both textual keyword and geometric information, the privacy protection of the trapdoors is not yet in place. Therefore, we face a challenge to design a comprehensive privacy-preserving scheme.

To address the aforementioned challenges, we propose an efficient privacy-preserving geographic keyword Boolean range

query (EPBRQ) scheme. Specifically, for challenge 1), we propose an index building method and design a recoding algorithm based on Bloom filters [21] to reduce the computational complexity by converting potential range-to-range comparisons into inner product calculations. Our scheme employs the secure Knn computation [15], an encryption technology that provides matrix obfuscation, as a solution to challenge 2), which has been widely proven in many studies [7], [9], [14] to protect the privacy of data well. Apparently, our scheme is different from [14] and [19], which are related to our scheme. Compared with [19], our scheme not only achieves advantages in the computational efficiency, but also eliminates the trapdoor index leakage problem brought by the employed encryption method. Different from [14], our scheme protects both textual keyword and geographic information, and our scheme is grid-based, with queries strictly following user-selected grid ranges without accuracy concerns.

Overall, our scheme focuses on computational efficiency and geographic information privacy and textual keyword privacy in spatial keyword queries. Our scheme not only solves the potential privacy leakage problem existing in previous schemes, but also achieves progress in computational efficiency compared with the state-of-the-art works. Our major contributions are summarized as follows.

- 1) We propose an efficient privacy-preserving geographic keyword Boolean range query (EPBRQ) scheme to support geometric range and textual keyword queries over encrypted spatial data, which has IND-SCPA security level and achieves *faster-than-linear* time complexity.
- 2) We construct a spatial grid-based index tree and design a recoding algorithm based on Bloom filters that can significantly improve computational efficiency by converting the range comparisons to secure inner product computations. We employ secure Knn computation to protect the privacy of data and queries, which also well supports fast and secure computation in our scheme.
- 3) We analyze the security of our scheme and conduct extensive experiments on real-world datasets to demonstrate that our scheme is more efficiency compared to existing schemes and is feasible on large-scale datasets.

The rest of this article is organized as follows. We outline the problem statement in the next section, and preliminaries in Section III. Then, we show the scheme details in Section IV. In Section V, we conduct a security analysis, and in Section VI, we give the experimental evaluation. We discuss related work in Section VII. Finally, Section VIII concludes this article.

II. PROBLEM STATEMENT

In this section, we start with a formal description of the target problem, then define the system model and threat model, and finally analyze the design goals of our scheme.

A. Problem Definition

The detailed expression of the problem of geographic keyword Boolean range query as is follows.

A data owner owns a spatial dataset $D = \{d_1, d_2, \dots, d_n\}$ distributed on a spatial grid, and $W = \{w_1, w_2, \dots, w_m\}$ denotes the keyword dictionary. Each data object $d_i (i \in [1, n])$ contains a geographic coordinate c and keywords $W_i = \{w_1, w_2, \dots, w_{m'}\} (W_i \subseteq W, m' \leq m)$. The owner uploads the spatial dataset to the cloud for other users to search. When a search user plans to do a query, he/she chooses k consecutive geographic coordinates from the spatial grid noted as R and p keywords from the keyword dictionary noted as W^* , and submits to the cloud also. The cloud server performs query service after receiving the request. Clearly, d_i will be returned if it satisfies: 1) $c \in R$; 2) for any keyword $w_j \in W^* (1 \leq j \leq k)$, there exists $w_j \in W_i$ else.

B. System Model

In our system, we consider three main types of entities, as shown in Fig. 1.

- 1) *Data owner*: Data owner generates an index Quadtree over the spatial data to ensure query efficiency, then encrypts the data and tree with corresponding encryption technologies, and finally uploads the encrypted spatial data and tree to the cloud server.
- 2) *Cloud server*: Cloud server provides storage services to the data owner. After receiving a query request, the cloud server executes search services and returns qualified results to the search user.
- 3) *Search user*: Authorized search user can generate trapdoors with authorization information and sends the trapdoors to the cloud server. After receiving the search results, he/she decrypts them with the symmetric secret key.

C. Threat Model

In our threat model, both data owners and search users are completely trustworthy. They encrypt data before uploading and generate trapdoors by convention, and do not actively reveal any real information to others. However, we treat the cloud server as a semihonest entity, also known as “*honest-but-curious*” [5], [6], [14], [19]. On the one hand, the cloud server will correctly provide storage and query services according to the protocol. On the other hand, it is eager to collect the sensitive user information at will without authorization. In practice, for example, some semihonest social platforms collect the users’ data and provide services for them, but also leak the sensitive information to gain more benefits. Hence, in this model, how to prevent the cloud server from “stealing” data is a challenge. To do this, we have to encrypt both data and queries to protect the privacy of data owners and data users, but also ensure that the encryption is searchable so that the cloud server can perform encrypted queries over encrypted data. Furthermore, we assume that data owners and search users share a secret key, so we also restrict the cloud server from collusion with users.

D. Design Goals

We define the design goals as follows.

TABLE I
NOTATION USED IN THIS ARTICLE

Symbols	Descriptions
T	Side length of the spatial grid
D	Spatial dataset, denotes as $D = \{d_1, \dots, d_n\}$
W	Keywords dictionary, denotes as $W = \{w_1, \dots, w_m\}$
W_i	Keywords of i th data object
g_i, b_i	Gray code and Bitmap of i th node
g'_i, b'_i	Bloom vector after recoding
$I, \tilde{I}$	Index Quadtree, denotes as $I = \{I_1, \dots, I_N\}$, and its encrypted form
R, W^*	Search geometric range and keywords
$T, \tilde{T}$	Search tokens, denotes as $T = \{tk_1, \dots, tk_k, b_{w^*}\}$, and its encrypted form

- 1) *Data privacy*: Data privacy includes geographic location privacy and textual keyword privacy, which may endanger users’ personal privacy if leaked. Therefore, both types of data need to be encrypted before uploading to prevent semihonest cloud servers.
- 2) *Query irrelevance*: The irrelevance of queries requires that the cloud server cannot distinguish whether two queries belong to the same query condition. Under this security requirement, the server cannot correlate the two queries and therefore cannot statistically compromise data privacy.
- 3) *Query efficiency*: Maintaining efficient query computation is a very important evaluation criterion for a system, especially in the era of Big Data, since employing encryption technology will bring higher computational overhead. Designing mechanisms to support more efficient queries is an ongoing research focus in encryption-based privacy-preserving research.

III. PRELIMINARIES

Before we introduce our detailed scheme, we first introduce some basic theoretical knowledge used in our work, such as Gray code [22], Bitmap, and secure Knn computation [15]. In addition, for easy reference, in Table I we summarize the main notation that will appear subsequently.

A. Gray Code and Bitmap

Gray code is a self-looping binary code in which any two adjacent values differ by only 1 b. The following defines the rapid generation process of a 1-D Gray code list, where “ $|$ ” is the concatenation operator and λ denotes the current recursive step:

$$G_1 = \{0, 1\}, G_\lambda = \{g_1, g_2, \dots, g_{2^\lambda}\}$$

$$G_{\lambda+1} = \{0|g_1, 0|g_2, \dots, 0|g_{2^\lambda}, 1|g_{2^\lambda}, 1|g_{2^\lambda-1}, \dots, 1|g_1\}.$$

Here is an example for $\lambda = 3$

$$G_1 = \{0, 1\}, G_2 = \{00, 01, 11, 10\}$$

$$G_3 = \{000, 001, 011, 010, 110, 111, 101, 100\}.$$

Bitmap is also a binary code, usually used to record or sort unsigned large datasets. Assuming m is the size of a textual keyword set, any subset can be represented by an m -bit binary

vector, where a “1” implies that the corresponding keyword exists in the subset and a “0” otherwise. Below, we enumerate a textual keyword set with $m = 8$ and give two subsets for demonstration

$$\begin{aligned} \text{Set} &: \{W_1, W_2, W_3, W_4, W_5, W_6, W_7, W_8\} \\ \text{Subset1} &: \{W_1, W_2, W_3, W_4\} \Rightarrow \text{“11110000”} \\ \text{Subset2} &: \{W_1, W_3, W_5, W_7\} \Rightarrow \text{“10101010”}. \end{aligned}$$

B. Secure Knn Computation

We adopt a secure Knn algorithm presented by Wong *et al.* [9] as the privacy-preserving scheme, which achieves secure inner computation by matrix obfuscation. In general, a secure Knn algorithm consists of four steps: **Init**, **GenIndex**, **GenTrap**, **Query**.

- 1) $(S, \mathbf{M}_1, \mathbf{M}_2) \leftarrow \text{Init}(1^\lambda)$: Enter a security parameter 1^λ , then the system randomly generates the secure key $\varphi = (S, \mathbf{M}_1, \mathbf{M}_2)$, where S denotes a d -dimensional binary vector and $\mathbf{M}_1, \mathbf{M}_2$ denote two $(d \times d)$ -dimensional invertible matrices.
- 2) $\tilde{v} \leftarrow \text{GenIndex}(S, \mathbf{M}_1, \mathbf{M}_2, v)$: For a d -dimensional vector v , split it into a pair of vectors (v_1, v_2) by S . Specifically, for $1 \leq j \leq d$, if $S[j] = 0$, $v_1[j] = v_2[j] = v[j]$. Otherwise, $v_1[j]$ and $v_2[j]$ are two random values that satisfy $v_1[j] + v_2[j] = v[j]$. Then output $\tilde{v} = (v_1 \mathbf{M}_1, v_2 \mathbf{M}_2)$.
- 3) $\tilde{w} \leftarrow \text{GenTrap}(S, \mathbf{M}_1, \mathbf{M}_2, w)$: Similarly, a d -dimensional vector w is first split into a pair of vectors (w_1, w_2) by S . However, the rules are slightly different, for $1 \leq j \leq d$, if $S[j] = 0$, $w_1[j]$ and $w_2[j]$ are randomly set to values that satisfy $w_1[j] + w_2[j] = w[j]$; otherwise, $w_1[j] = w_2[j] = w[j]$. Then output $\tilde{w} = \mathbf{M}_1^{-1} w_1, \mathbf{M}_2^{-1} w_2$.
- 4) $v \cdot w \leftarrow \text{Query}(\tilde{v}, \tilde{w})$: After receiving $\tilde{v}$ and $\tilde{w}$, calculate the inner product according to the following equation:

$$\begin{aligned} \tilde{v} \cdot \tilde{w} &= (v_1 \mathbf{M}_1, v_2 \mathbf{M}_2) \cdot (\mathbf{M}_1^{-1} w_1, \mathbf{M}_2^{-1} w_2)^T \\ &= v_1 \mathbf{M}_1 \cdot \mathbf{M}_1^{-1} w_1 + v_2 \mathbf{M}_2 \cdot \mathbf{M}_2^{-1} w_2 \\ &= v_1 \cdot w_1 + v_2 \cdot w_2 \\ &= v \cdot w. \end{aligned}$$

IV. PROPOSED SCHEME

We propose an efficient privacy-preserving geographic keyword Boolean range query (EPBRQ) scheme to support geometric range and keyword queries over encrypted spatial data. We will introduce the EPBRQ in detail from the following sections.

A. System Initialization

Select a secure parameter λ , system randomly generates the secret key $\varphi = (msk, S, \mathbf{M}_1, \mathbf{M}_2)$, where msk is the symmetric key used to encrypt the original spatial data. S , and $\mathbf{M}_1, \mathbf{M}_2$ are employed to work with the vector indexes, presenting a d -dimensional binary vector for partitioning and two $(d \times d)$ -dimensional invertible matrices for encryption, respectively. Here, we vaguely define the length of the secret key as d , and

more details on how to determine the value of d will be discussed at the end of this section.

B. Index Quadtree Building

We have designed a new index construction method that takes into account both keywords and geometric ranges. We predefine the dataset as $D = \{d_1, d_2, \dots, d_n\}$ and the keyword dictionary as $W = \{w_1, w_2, \dots, w_m\}$.

Fig. 2 illustrates an index building process, which consists of three phases. In phase 1, given a $T \times T$ spatial grid, the core work is to replace the digital axes with $\lceil \log_2 T \rceil$ -bit Gray code sequences to generate a *Gray code grid*. Then in phase 2, encode the keywords of each data object $d_i (1 \leq i \leq n)$ into a m -bits bitmap $b_i \leftarrow \text{Bitmap}(W_i)$, as explained in Section III-A, and map the data objects into the corresponding *Gray code grid* cells according to their geographic coordinates. Finally, in phase 3, an index Quadtree [23] will be build based on the *Gray code grid*. Specifically, each spatial range node will generate its own Gray code and bitmap through the following two operations.

- 1) Vector merging between bitmaps: The essence of vector merging is to perform *bit-or* operations between binary vectors. For example, “10101000” and “10010010” get “10111010,” which is recoded as “10101000” $\vee$ “10010010” = “10111010”. The i th spatial range node will generate its bitmap with $(b_1 \vee b_2 \vee \dots \vee b_{i'})$, where $b_j (1 \leq j \leq i')$ is the bitmap of j th child node.
- 2) Vector compression between Gray codes: The essence of vector compression is to compress two binary vectors with only one bit difference into a single vector with wildcard “*.” For example, “0000” and “0001” get “000*,” which is recoded as “0000” $\wedge$ “0001” = “000*.” Then, the i th spatial range node will generate its Gray code with $(g_1 \wedge g_2 \wedge \dots \wedge g_{i'})$, where $g_j (1 \leq j \leq i')$ is the Gray code of j th child node.

We have shown more examples of vector compression in Fig. 3. Although vector compression restricts the difference in only one bit between vectors, due to the characteristics of the Gray code, each spatial range node can successfully perform the compression operation and finally successfully build an index Quadtree. We formally represent this tree as $I = \{I_1, I_2, \dots, I_N\}$, where N denotes the number of nodes and $I_i = \{g_i, b_i\} (1 \leq i \leq N)$.

C. Index Recoding Algorithm and Encryption

The search on above index tree needs frequent comparisons between Gray codes with wildcards to fix the intersection range. However, to the best of our knowledge, most of existing encryption techniques do not directly support such comparisons in ciphertext. Although some equivalent methods, such as in [19], have been proposed to deal with this problem indirectly, they tend to incur higher computational consumption. To resolve this problem, we design a recoding algorithm based on Bloom filters, by which the comparison of the Gray code with wildcards is converted into the inner product computation by Knn encryption. The recoding algorithm can be briefly described as: for an n -bit vector $v = \{v_1, v_2, \dots, v_n\}$, map each bit $v_i (1 \leq i \leq n)$ into an

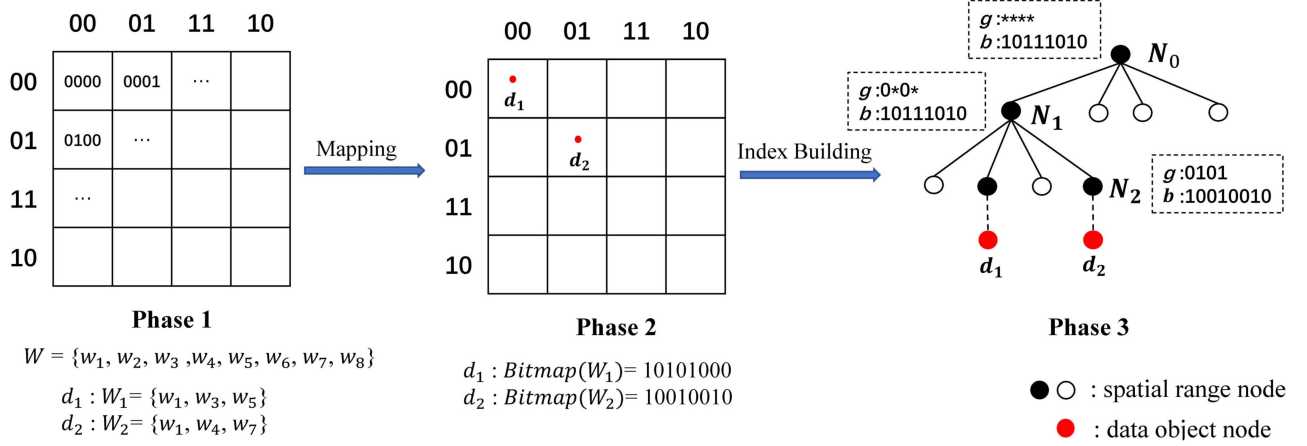


Fig. 2. Demonstration of the index construction work, which consists of three phases and results in an index Quadtree.

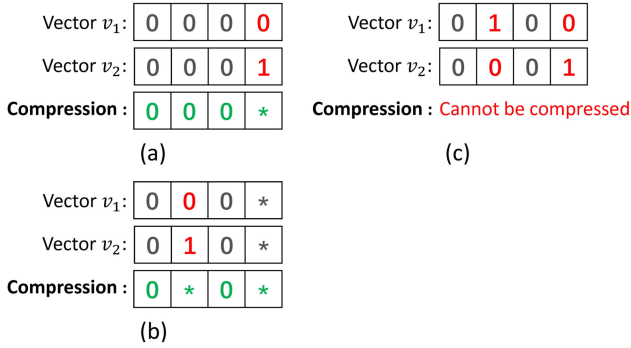


Fig. 3. More vector compression examples.

l -b Bloom filter bf_i , and finally concatenate all the Bloom filters into an $(l \times n)$ -bit Bloom vector $bv = (bf_1 | bf_2 | \dots | bf_n)$. The detail recoding procedure is illustrated in *Algorithm 1*, where l represents the length of a Bloom filter. We do not need to consider the false positives of the Bloom filter since each filter contains only one element. To prevent the bit 0 and 1 in the same position from being mapped to the same Bloom filter, in addition, we define the Bloom filter of bit 0 as $bf_{standard}$ and the filter of bit 1 is modified by $bf_{standard}$. For example, for bit 0, we assume that $bf_{standard} = "0101."$ Then for bit 1, we find the nearest $x_1 = 1$ such that $bf_{standard}[x_1] = 1$ and $x_2 = 0$ such that $bf_{standard}[x_2] = 0$ and update $bf_{standard}[x_1] = 0$ and $bf_{standard}[x_2] = 1$ to get the Bloom vector "1001." With this algorithm, each node updates its Gray code g to g' and bitmap b to b' . After that, we encrypt the spatial dataset D with the symmetric key msk , and then encrypt the index Quadtree with secure Knn computation. Specifically, for the i th node I_i , divide its g'_i into $(g'_{i,1}, g'_{i,2})$ as described in Section III-C.

- 1) If $S[j] = 0$, then $g'_{i,1}[j] = g'_{i,2}[j] = g'_i[j]$.
- 2) If $S[j] = 1$, then $g'_{i,1}[j]$ and $g'_{i,2}[j]$ are two random values satisfying $g'_{i,1}[j] + g'_{i,2}[j] = g'_i[j]$, where $1 \leq j \leq |S|$. Similarly, b'_i is also split into $(b'_{i,1}, b'_{i,2})$ as described above. Finally, we get the encryption format $\tilde{I}_i =$

Algorithm 1: Recoding Algorithm.

Input: vector $v = \{v_1, v_2, \dots, v_i\}$, hash functions $H = \{h_1, h_2, \dots, h_j\}$ and the initial length l of each Bloom filter
Output: a Bloom vector

- 1: Initialise $BF_{set} \leftarrow []$,
- 2: **for each** $index \in [1, i]$ **do**
- 3: Initialise $bf_{standard} \leftarrow 0^l$;
- 4: **if** $v_{index} = *$ **then**
- 5: $BF_{set}.add(bf_{standard})$;
- 6: **else**
- 7: **for** $h' \in H$ **do**
- 8: Set $bf_{standard}[h'(0|index)\%l] \leftarrow 1$;
- 9: **end for**
- 10: **if** $v_{index} = 0$ **then**
- 11: $BF_{set}.add(bf_{standard})$;
- 12: **else**
- 13: Find nearest x_1 such as $bf_{standard}[x_1] = 1$;
- 14: Find nearest x_2 such that $bf_{standard}[x_2] = 0$;
- 15: Set $bf_{standard}[x_1] \leftarrow 0, bf_{standard}[x_2] \leftarrow 1$;
- 16: $BF_{set}.add(bf_{standard})$;
- 17: **end if**
- 18: **end if**
- 19: **end for**
- 20: Initialise BV as output;
- 21: **for** $bf \in BF_{set}$ **do**
- 22: $BV \leftarrow BV | bf$
- 23: **end for**
- 24: **return** BV ;

$\{(g'_{i,1}M_1, g'_{i,2}M_2), (b'_{i,1}M_1, b'_{i,2}M_2)\}$ for I_i and write the encrypted index Quadtree as $\tilde{I}$.

D. Trapdoor Generation

Query privacy protection is also a crucial part of our system. A search user generates special search trapdoors so that the cloud server can perform a query on the encrypted indexes. To initiate a

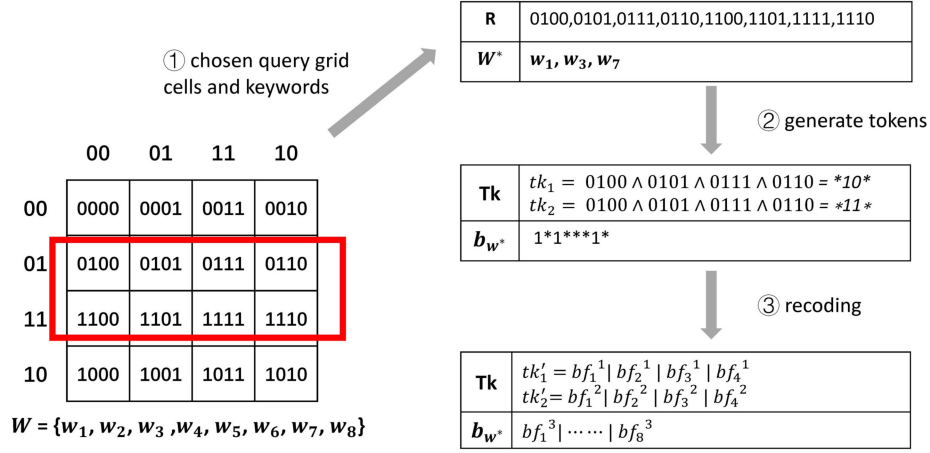


Fig. 4. Example of trapdoor generation process without encryption, where bf generally represent l -bits bloom filters.

query request, a search user marks a region on the *Gray code grid* as R and specifies willingness query keywords from W as W^* . Next, he/she generates the keyword token $b_{w^*} \leftarrow \text{Bitmap}(W^*)$ and immediately sets all 0s to *. Moreover, to reduce the number of range tokens, vector compression operations are performed as much as possible between the Gray codes in R . Then, the search tokens $T = \{tk_1, tk_2, \dots, tk_k, b_{w^*}\}$ are loaded. To maintain efficient query, each token will also be recoded into a Bloom vector. The detailed implementation of the trapdoor recoding algorithm is approximately the same as Algorithm 1, but the two lines marked need to be modified as follows:

- ① $bf_{standard}$ will be initialized with 1^l instead 0^l ;
- ② set $bf_{standard}[h'(0||index)\%m]$ to 0 instead 1.

With this modified algorithm, $tk_i (i \in [1, k])$ and b_{w^*} are expanded into tk'_i and b'_{w^*} , respectively. We use Fig. 4 to simulate the trapdoor generation process in a 4×4 *Gray code grid*, where the red box represents the user-selected query region. After recoding, then, each token needs to be encrypted also, where $tk'_i (i \in [1, k])$ is split into $(tk'_{i,1}, tk'_{i,2})$ and b'_{w^*} is split into $(b'_{w^*,1}, b'_{w^*,2})$. The splitting rules for tk'_i are shown as follows.

- 1) If $S[j] = 1$, then $tk'_{i,1}[j] = tk'_{i,2}[j] = tk'_i[j]$.
- 2) If $S[j] = 0$, then $tk'_{i,1}[j]$ and $tk'_{i,2}[j]$ are two random values satisfying $tk'_{i,1}[j] + tk'_{i,2}[j] = tk'_i[j]$

where $1 \leq j \leq |S|$ and the same operations applies to b'_{w^*} . Finally, the split vector pairs are multiplied with the inverse matrices M_1^{-1}, M_2^{-1} to obtain the final ciphertext format $\tilde{T} = \{tk_1, tk_2, \dots, tk_k, (M_1^{-1}b'_{w^*,1}, M_2^{-1}b'_{w^*,2})^T\}$, where $tk_i = (M_1^{-1}tk'_{i,1}, M_2^{-1}tk'_{i,2})^T (i \in [1, k])$.

E. Depth-First Query Over Index Quadtree

After receiving a query request, the cloud server starts traversing the encrypted index Quadtree with the following rules.

- 1) For a spatial range node, both range and keyword judgments are required, and if true, access its children nodes. Otherwise backtrack to its parent node, if it exists.

- 2) For a data object node, only the bitmap calculation is required, and the satisfied data object will be put into the returned results.

The entire query process is described in *Algorithm 2*. For a clearer understanding of the ciphertext computation process of a node, we take the i th spatial node $\tilde{I}_i$ as an example. First, we execute keywords comparison, which actually is an inner product between $\tilde{b}_i$ and $\tilde{b}_{w^*}$, expressed as

$$\begin{aligned} \tilde{b}_i \cdot \tilde{b}_{w^*} &= (b'_{i,1}M_1, b'_{i,2}M_2) \cdot (M_1^{-1}b'_{w^*,1}, M_2^{-1}b'_{w^*,2})^T \\ &= b'_{i,1}M_1 \cdot M_1^{-1}b'_{w^*,1} + b'_{i,2}M_2 \cdot M_2^{-1}b'_{w^*,2} \\ &= b'_{i,1} \cdot b'_{w^*,1} + b'_{i,2} \cdot b'_{w^*,2} \\ &= b'_i \cdot b'_{w^*}. \end{aligned}$$

If the calculation result is 0, then move to the next step, which is to perform k times inner product between $\tilde{g}_i$ and $tk_i (i \in [1, k])$, as

$$\begin{aligned} \tilde{g}_i \cdot tk_i &= (g'_{i,1}M_1, g'_{i,2}M_2) \cdot (M_1^{-1}tk'_{i,1}, M_2^{-1}tk'_{i,2})^T \\ &= g'_{i,1}M_1 \cdot M_1^{-1}tk'_{i,1} + g'_{i,2}M_2 \cdot M_2^{-1}tk'_{i,2} \\ &= g'_{i,1} \cdot tk'_{i,1} + g'_{i,2} \cdot tk'_{i,2} \\ &= g'_i \cdot tk'_i. \end{aligned}$$

If there exists a $\tilde{g}_i \cdot tk_j$ that is 0, it means that the the space range represented by the current node has a intersection with the query range, then all its child nodes are accessed. When a data object leaf node is accessed, only the keyword comparison needs to be computed. The data corresponding to the nodes that satisfies the requirements are returned as the results.

Correctness: In order to prove the correctness of our inference, we need to show that $b'_i \cdot b'_{w^*} = 0$ holds when the case is true. In Section IV-C, we have mentioned that b'_i is spliced together by m Bloom filters, which can be written as $\{bf_1|bf_2|\dots|bf_m\}$. We also write b'_{w^*} as $\{bf_1^*|bf_2^*|\dots|bf_m^*\}$

Algorithm 2: Depth-First Query Over Index Quadtree.

Input: root node N_0 of the index Quadtree and search tokens $\tilde{Q} = \{tk_1, tk_2, \dots, tk_k, b_{w^*}\}$
Output: return results I^*

```

1: Initialise Queue  $\leftarrow [N_0]$ ,  $I^* = []$ ;
2: while Queue is not null do
3:   Initialise state = FALSE;
4:   Set  $node = \text{Queue.pop}()$ 
5:   if is data object node &  $\text{Knn.Query}(node.b, \tilde{b}_{w^*}) = 0$  then
6:      $I^*.add(node)$ 
7:   else
8:     if  $\text{Knn.Query}(node.b, \tilde{b}_{w^*}) = 0$  then
9:       for each  $tk_j \in (tk_1, tk_2, \dots, tk_k)$  do
10:        if  $\text{Knn.Query}(tk_j, node.g) = 0$  then
11:          state = TRUE
12:          break;
13:        end if
14:      end for
15:    end if
16:    if state = TRUE then
17:      Queue.push(all child nodes of  $node$ )
18:    end if
19:  end if
20: end while
21: return  $I^*$ ;

```

and get

$$b'_i \cdot b'_{w^*} = \sum_{k=1}^m b_{f_k} \cdot b_{f_k^*}.$$

In the case where $b'_i(b'_{w^*})[k] = *$, we have $b_{f_k}(b_{f_k^*}) = 0$ and $b_{f_k} \cdot b_{f_k^*} = 0$ holds. And as for the case where $b'_i(b'_{w^*})[k] = 0(1)$, we have $b_{f_k} = \sim b_{f_k^*}$ and $b_{f_k} \cdot b_{f_k^*} = 0$ still holds. Therefore, we can get that if two keyword sets match condition, their bitmap inner product is $b'_i \cdot b'_{w^*} = 0$. The same derivation process is also applicable to the calculation of g'_i and tk'_i , we will not elaborate it again due to limited space.

Discuss: A general consensus is that the strength of a 1024-b RSA key is roughly equivalent to that of an 80-b symmetric key [15]. It means that the length of the secret key affects the security level of the symmetric encryption. Therefore, in the *system initialization* phase, we must choose an appropriate secret key length d to ensure the security level of our system. The length of d is actually related to the Bloom vector recoded by Gray code and the bitmap, for example, for recoded bitmap, $d = m \times l$. As m is fixed, we can achieve different key security levels by adjusting the size of l . In our scheme, we predefine a key security level parameter l_{level} and calculate the bloom filter size $l_g = \lceil \frac{l_{level}}{\log_2 T} \rceil$ of Gray code and $l_b = \lceil \frac{l_{level}}{m} \rceil$ of bitmap, respectively, for recoding. Note, however, that the value of l cannot be lower than 2, since at least two bits are needed to distinguish between $*$ and $\{0,1\}$.

V. SECURITY ANALYSIS

In this section, based on [19], we formally analyze the security of our scheme. We give some security definitions and prove that EPBRQ is IND-SCPA secure, which is the relaxation of the standard IND-CPA but may leak the message the attacker to commit in advance [24]. In practice, IND-SCPA secure is widely used in [10]–[12] and [19].

A. Security Definitions

Below we define four allowable leaks that exist in most searchable encryption schemes.

- 1) *Size pattern*: The cloud server knows the scale of the data and indexes stored, the number of trapdoors received, and the size of the returned result.
- 2) *Access pattern*: The cloud server knows which identifiers of encrypted data are touched and which identifiers satisfy a specific query request.
- 3) *Search pattern*: The cloud server knows the number of the same encrypted data that a geometric range query has related to a previous query.
- 4) *Path pattern*: The cloud server knows the access path on the tree structure.

We use $\mathcal{L}(D, Q)$ to denote the leaks above, where D represents the given database and Q represents a submitted query request. And in the following security game, $\mathcal{L}$ stands for the leaks allowed in a round.

Definition 1 (IND-SCPA index privacy of EPBRQ): Here, $\Pi = (\text{KeyGen}, \text{IndexBuild}, \text{TrapGen}, \text{Query})$ represents a probabilistic EPBRQ scheme based on the data space Δ_T^d and the security parameter λ . We define the security game between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$ as follows.

- 1) *Init*: Adversary $\mathcal{A}$ creates two databases of index $D_0 = \{D_{0,1}, \dots, D_{0,n}\}$ and $D_1 = \{D_{1,1}, \dots, D_{1,n}\}$ with the same number of index records and sends them to the challenger, where $D_{0,i}, D_{1,i} \in \Delta_T^d$, for $1 \leq i \leq n$.
- 2) *Setup*: Challenger $\mathcal{C}$ executes $\text{KeyGen}(1^\lambda)$ to obtain the system secret key $\varphi = \{S, M_1, M_2\}$ and keeps it private.
- 3) *Phase I*: Adversary $\mathcal{A}$ purposefully selects some requests from the following types to submit to the challenger.
 - a) *IndexBuild request*: On the j th IndexBuild request, $\mathcal{A}$ randomly creates a database $D'_j = \{D'_{j,1}, D'_{j,2}, \dots, D'_{j,n}\}$, $D'_{j,i} \in \Delta_T^d$ ($1 \leq i \leq n$). Then challenger responses this request with encrypted index $\tilde{I}_j \leftarrow \text{IndexBuild}(D'_j, \varphi)$.
 - b) *TrapGen request*: On the j th TrapGen request, $\mathcal{A}$ randomly selects a geographic keyword range query $Q_j = \{R_j, W_j^*\}$, where $R_j \in \Delta_T^d$, $W_j^* \in W$. Challenger $\mathcal{C}$ answers this request with encrypted trapdoor $\tilde{Q}_j \leftarrow \text{TrapGen}(Q_j, \varphi)$, where Q_j satisfies $\mathcal{L}(D_0, Q_j) = \mathcal{L}(D_1, Q_j)$.
- 4) *Challenge*: With D_0, D_1 chosen in Init, the challenger flips a coin to decide $b \in (0, 1)$, and returns $\tilde{I}_b$ to $\mathcal{A}$, where $\tilde{I}_b \leftarrow \text{IndexBuild}(D_b, \varphi)$.

- 5) *Phase 2*: In this phase, $\mathcal{A}$ can still send some requests to $\mathcal{C}$ purposefully, but these requests also need to meet the type requirements in Phase 1.
- 6) *Guess*: $\mathcal{A}$ takes a guess b' of b .

If the advantages of any potential polynomial-time adversary can be ignored in the above security game, then in terms of index privacy, Π has sufficient resistance to selective chosen-plaintext attacks

$$\text{Adv}_{\Pi, \mathcal{A}}^{\text{IND-SCPA-Index}}(1^\lambda) = |\Pr[b' == b] - \frac{1}{2}| \leq \text{negl}(1^\lambda).$$

Definition 2 (IND-SCPA trapdoor privacy of EPBRQ): Here, $\Pi = (\text{KeyGen}, \text{IndexBuild}, \text{TrapGen}, \text{Query})$ represents a probabilistic EPBRQ scheme based on the data space Δ_T^d and the security parameter λ . We define the security game between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$ as follows.

- 1) *Init*: Adversary $\mathcal{A}$ sends two plaintexts of geographic keyword range queries $Q_0 = \{R_0, W_0^*\}$ and $Q_1 = \{Q_1, W_1^*\}$ to the challenger $\mathcal{C}$, where $R_0, R_1 \in \Delta_T^d$ and $W_0^*, W_1^* \in W$.
- 2) *Setup*: Challenger $\mathcal{C}$ executes $\text{KeyGen}(1^\lambda)$ to obtain the system secret key $\varphi = \{S, \mathbf{M}_1, \mathbf{M}_2\}$ and keeps it private.
- 3) *Phase 1*: Adversary $\mathcal{A}$ purposefully selects some requests from the following types to submit to the challenger.
 - a) *IndexBuild request*: On the j th IndexBuild request, $\mathcal{A}$ randomly creates a database $D_j = \{D_{j,1}, D_{j,2}, \dots, D_{j,n}\}$, $D_{j,i} \in \Delta_T^d (1 \leq i \leq n)$. Then challenger responses with encrypted index $\tilde{T}_j \leftarrow \text{IndexBuild}(D_j, \varphi)$, where D_j satisfies $\mathcal{L}(D_j, Q_0) = \mathcal{L}(D_j, Q_1)$.
 - b) *TrapGen request*: On the j th TrapGen request, $\mathcal{A}$ randomly selects a geographic keyword range query $Q_j = \{R_j, W_j^*\}$, where $R_j \in \Delta_T^d$, $W_j^* \in W$. Challenger $\mathcal{C}$ answers this request with encrypted trapdoor $\tilde{Q}_j \leftarrow \text{TrapGen}(Q_j, \varphi)$.
- 4) *Challenge*: With Q_0, Q_1 chosen in Init, the challenger flips a coin to decide $b \in (0, 1)$, and returns $\tilde{Q}_b$ to $\mathcal{A}$, where $\tilde{Q}_b \leftarrow \text{TrapGen}(Q_b, \varphi)$.
- 5) *Phase 2*: In this phase, $\mathcal{A}$ can still send some requests to $\mathcal{C}$ purposefully, but these requests also need to meet the type requirements in Phase 1.
- 6) *Guess*: $\mathcal{A}$ takes a guess b' of b .

If the advantages of any potential polynomial-time adversary can be ignored in the above security game, then in terms of index privacy, Π has sufficient resistance to selective chosen-plaintext attacks

$$\begin{aligned} \text{Adv}_{\Pi, \mathcal{A}}^{\text{IND-SCPA-Trapdoor}}(1^\lambda) \\ = |\Pr[b' == b] - \frac{1}{2}| \leq \text{negl}(1^\lambda). \end{aligned}$$

B. Security Analysis

We analyze the security of our EPBRQ scheme below. Since EPBRQ is encrypted by secure Knn computation, we can prove that our scheme is also IND-SCPA by proving that Knn is IND-SCPA.

TABLE II
NOTATION FOR COMPARE ANALYSIS

Notations	Descriptions
n	Size of dataset
n'	Size of spatial nodes in index tree
m	Size of keyword dictionary
k	Number of range tokens
v	Number of spatial vector instances
v'	Number of index instances
T_e	Time taken to compute a exponentiation
T_p	Time taken to compute a pairing
T_{bloom}	Time taken to generate a bloom filter
T_{matrix}	Time taken to perform a matrix product
T_{div}	Time taken to perform a vector split
T_{sum}	Time taken to perform a vector sum
T_{prf}	Time taken to compute a pseudo random function
T_{xor}	Time taken to perform an or operation
T_{enc}	Time taken to perform a symmetric encryption
T_{dec}	Time taken to perform a symmetric decryption

Theorem 1 (Index privacy of EPBRQ): Our EPBRQ scheme is IND-SCPA index secure, as long as secure Knn computation is IND-SCPA secure.

Proof 1: We simulate the security game in Definition 1 with an adversary $\mathcal{A}$ from the security game of Knn to prove our EPBRQ is IND-SCAP index secure. And then we transfer the proof of EPBRQ to the proof of KNN. A proof of IND-CLS-CPA index secure for Knn was completed in [14], and similarly, the proof of IND-SCAP index secure for Knn can be substituted into that work as well. Considering the space limitation, we will not elaborate on it. Eventually, we have

$$\begin{aligned} \text{Adv}_{\text{EPBRQ}, \mathcal{A}}^{\text{IND-SCPA-Index}}(1^\lambda) \\ = 1 - (1 - \text{Adv}_{\text{Knn}, \mathcal{A}}^{\text{IND-SCPA-Index}}(1^\lambda))^n \\ \leq \text{negl}(1^\lambda). \end{aligned}$$

Therefore, our scheme is IND-SCPA index secure, as long as Knn is IND-SCAP secure also.

Theorem 2 (Trapdoor privacy of EPBRQ): Our EPBRQ scheme is IND-SCPA trapdoor secure, as long as secure Knn computation is IND-SCPA secure.

Proof 2: The IND-SCPA trapdoor privacy of our EPBRQ can be proved using the same as the procedure in *Proof 1*, so the description is not repeated here.

VI. PERFORMANCE ANALYSIS

First, we establish mathematical formulas for each stage of our scheme to theoretically analyze the performance. Then we evaluate each stage based on the experimental results to show that our work is feasible.

A. Theoretical Analysis

Consider a Quadtree based on a $T \times T$ spatial grid, the geometric index of each node is $2\lceil \log_2 T \rceil$ -bits. With this parameter and notation in Table II, we evaluate the computation cost of GenIndex, GenTrap, and Query below.

- 1) *GenIndex*: For a spatial node, it costs $(2\lceil \log_2 T \rceil + m)T_{bloom}$ in recoding phase and $T_{div} + 2T_{matrix}$ in

TABLE III
COMPARISON OF COMPUTATIONAL COMPLEXITY

Scheme	GenIndex	GenTrap	Query
FastGeo	$O(v)((6T+2)T_e + T_{prf})$	$O(v')(8T_e + T_{prf})$	$O(v')((2T+2)T_p)$
PBRQ-L	$O(n)(2\lceil\log_2 T\rceil + m)T_{prf}$	$O(k)((2\lceil\log_2 T\rceil + m)T_{prf} + T_{enc})$	$O(kn)((2\lceil\log_2 T\rceil + 2)mT_{xor} + T_{dec})$
PBRQ-Q	$O(n)(mT_{prf} + T_{enc} + mT_{prf})$	$O(k)(2\lceil\log_2 T\rceil + m)(T_{prf} + T_{xor}) + O(t+1)T_{enc}$	$O(k(2\lceil\log_2 T\rceil + 1)((2\lceil\log_2 T\rceil + m)T_{xor} + T_{dec}))$
EPBRQ	$O(n)(mT_{bloom} + T_{div} + 2T_{matrix}) + O(n')((2\lceil\log_2 T\rceil + m)T_{bloom} + T_{div} + 2T_{matrix})$	$O(k+1)(2T_{matrix} + T_{div}) + O(k)(2\lceil\log_2 T\rceil + m)T_{bloom}$	$O(k(\lceil\log_2 T\rceil + 1) + 1)(2T_{matrix} + 2T_{sum})$

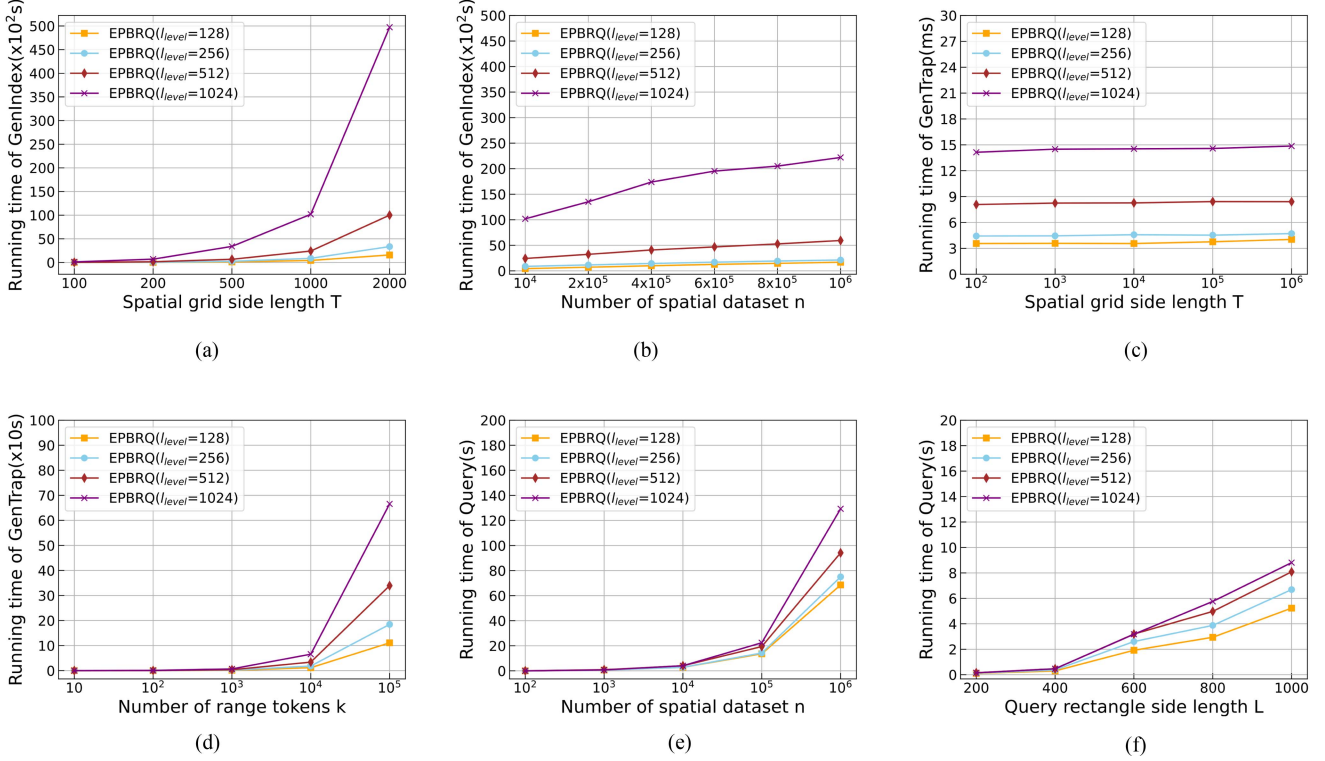


Fig. 5. Experimental results. (a) and (b) Computational cost of *GenIndex* in different variables. (c) and (d) Computational cost of *GenTrap* in different variables. (e) and (f) Computational cost of *Query* in different variables.

encryption phase. As for a data object node, it only has keyword property, so its computation cost is mT_{bloom} and $T_{div} + 2T_{matrix}$, respectively. The final cost under n data object node and n' spatial data is $O(n)(mT_{bloom} + T_{div} + 2T_{matrix}) + O(n')((2\lceil T \rceil + m)T_{bloom} + T_{div} + 2T_{matrix})$.

- 2) *GenTrap*: Search tokens include k range tokens and one keyword token. Similarly, each range token spends $2\lceil\log_2 T\rceil T_{bloom}$ and $T_{div} + 2T_{matrix}$ in recoding phase and encryption phase, while keyword token spends mT_{bloom} and $T_{div} + 2T_{matrix}$. The final cost is $O(k+1)(T_{div} + 2T_{matrix}) + O(k)(2\lceil T \rceil + m)T_{bloom}$.
- 3) *Query*: We assume that only one node at each level satisfies the query request, including $\lceil\log_2 T\rceil + 1$ spatial nodes and one data object node. A spatial node spends $k(2T_{matrix} + 2T_{sum})$ in the query phase while a data object node spends $2T_{matrix} + 2T_{sum}$. So, the final query complexity is $O(k(\lceil\log_2 T\rceil + 1) + 1)(2T_{matrix} + 2T_{sum})$.

TABLE IV
NUMBER OF RANGE TOKENS

Side length L	200	400	600	800	1000
$\tilde{k}$	62.53	78.11	98.06	110.59	178.25

We collect the computation cost of our work with some existing schemes in Table III.

B. Experimental Evaluations

We will analyze *GenIndex*, *GenTrap*, and *Query* overheads of our scheme in Fig. 5 when l_{level} is 128, 256, 512, and 1024, respectively, which are the common key lengths used in symmetric encryption schemes. More comparative analyses with existing work are presented in Table IV, where the security parameter λ in PBRQ-L [19] and PBRQ-Q [19] is 128 and

the encryption scheme in FastGeo [10] is implemented with the pairing-based cryptography (PBC) library. All test data are selected from Gowalla dataset, which contains 6 442 890 check-in data objects containing location coordinates collected from 196 591 users. This dataset is also the test dataset for our comparison schemes [19] and [10]. Since the data objects in the Gowalla dataset do not have keyword attributes, we generate a keyword dictionary of size $m = 100$ from Enron dataset and assign ten keywords to each data object randomly. We write the programs in Python3 and run in Linux with 8 Intel Xeon Silver 4110 CPU, 128 RAM. To ensure reliability, each running time is the average of multiple experiments.

GenIndex overhead: To reduce the time overhead, we build a noncomplete Quadtree instead of a complete Quadtree in *GenIndex* phase, where the grid cells that do not contain data are discarded. We draw the time curve under different spatial grid side length T in Fig. 5(a), where dataset size $n = 10\,000$. It can be seen that for the same T , the higher the l_{level} , the higher the overhead of each encryption and the longer the index building time. Fig. 5(b) shows the runtime of *GenIndex* on dataset sizes from 10^4 to 10^6 , where grid side length $T = 1000$. Obviously, the larger the data size and the wider the data distribution domain, the more nodes there will be in the index tree, which naturally leads to longer runtime. Although the tree building overhead is still high even after optimization, we consider it acceptable since *GenIndex* is executed performed only once, while queries are the frequent user operations.

GenTrap overhead: As mentioned in Section IV-D, a search request includes k range tokens and one keyword token. In Fig. 5(c), we fix $k = 1$ and keep increasing the size of T . As can be seen from the curve, the change in running time is small for the same l_{level} although T rises from 10^2 to 10^6 . This is due to the exponential growth property of the Gray code, where the bit length of the range token actually only grows from $2 \times \lceil \log_2 10^2 \rceil = 14$ to $2 \times \lceil \log_2 10^6 \rceil = 40$, which has very little impact on the time overhead. In Fig. 5(d), we fix $T = 1000$ and increase k from 10 to 10^5 , and plot the results. In the same l_{level} , the running time is positively correlated with the value of k , especially when k exceeds 10^4 . Fortunately, Table IV, where L denotes a $L \times L$ rectangle and $\tilde{k}$ denotes the average number of range tokens after 100 experiments, has shown that k is actually difficult to reach 10^3 or even 10^4 due to the high compressibility of the Gray code. While there exist some unusually complex shapes that would result in a large k , they have little practical application. So we think the time overhead in *GenTrap* is within an acceptable range.

Query overhead: In Fig. 5(e), we fix query area and $T = 1000$, and keep increasing the dataset size n . Unlike *GenIndex* and *GenTrap*, there is no significant multiplier gap between the running times of different l_{level} . The same case is found in Fig. 5(f), where we fix $T = 1000$ and $n = 10\,000$ and increase query area from 200×200 to 1000×1000 , with little difference in time cost for different l_{level} . This is because the matrix multiplication complexity in Knn is slowly affected by the growth of the key length. We test Knn.Query() with different secret key lengths in Fig. 6, and the time only changes from $7 \times 10^{-6}s$ to $11 \times 10^{-6}s$ while the key length increases from

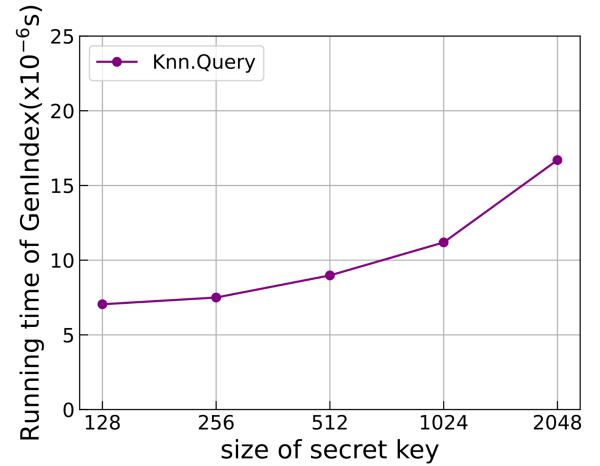


Fig. 6. Running time of Knn.Query() with different key lengths.

TABLE V
COMPARISON AMONG SCHEMES

n	2×10^4	4×10^4	6×10^4	8×10^4	10^5
FastGeo	405.73 s	660.23 s	946.24 s	1129.75 s	1293.12 s
PBRQ-L	59.18 s	100.18 s	141.03 s	174.53 s	210.26 s
PBRQ-Q	14.88 s	21.98 s	30.7 s	37.15 s	43.56 s
EPBRQ	7.15 s	12.42 s	16.01 s	18.53 s	23.51 s

128 to 1024. In other words, our scheme maintains efficient query operations with a high key security level l_{level} .

Finally, we compare the time overhead of the query cycle, including both *Genindex* and *Query* phases, between our EPBRQ scheme and the previous schemes on the test dataset. We fix the query area as a 500×500 rectangle with grid side length $T = 1,000$ and increase the dataset size n from 2×10^4 to 10^5 . We record the time overhead in Table V, and the final result shows that our scheme achieves progress in query efficiency, almost x2 faster than the state-of-the-art work.

VII. RELATED WORK

A. Keyword Privacy

The privacy concerns in cloud computing have prompted the research on secure keyword search. The earliest attempt at searchable encryption was done by Song *et al.* [4] in which the server was allowed to perform single keyword queries over encrypted files, but at a high computational cost. To accelerate the search, Goh *et al.* [25] proposed a scheme of creating a keyword index for each file so that the server only needs to traverse the index rather than the entire file when searching. In the following work, Curtmola *et al.* [26] gave the first definition of access pattern and search pattern, and used hash tables as an alternative to searchable encryption. Wang *et al.* [27] first defined the problem of secure keyword ranked search on encrypted cloud data and designed an inverted index structure to speed up the queries. To be closer to the real scene, Cao *et al.* [28] first extended the secure keyword search for multikeyword queries. Their approach vectorized the list of file keywords and then applied matrix multiplication to implement top- k related data

file queries. Zhang *et al.* [5] further considered ranked multi-keyword search with multiple data owners, where different data owners encrypt their files with different secret keys. A more comprehensive multikeyword multiuser scheme was proposed by Shu *et al.* [9]. They utilize the proxy reencryption technique to allow each user to have their own independent secret key.

All studies have contributed significantly to the development of keyword privacy-preserving. However, these studies only focus on keyword queries, which cannot be directly applied to solve the geographic keyword Boolean range query problem.

B. Location Privacy

The study most closely related to our work is location privacy protection in geometric range query. In principle, homomorphic encryption (HE) [29] supports secure ciphertext computation, but also presents an unacceptable computational overhead. Without HE, Wang *et al.* [11] proposed a circular range search scheme that divides the query area into multiple concentric circles to retrieve data. Zhu *et al.* [30] also proposed a scheme for circular range search over encrypted spatial data. Unfortunately, these two solutions are only valid for circles and not for other geometric areas. An improved scheme was proposed in [12], Wang *et al.* extended the scheme [11] to arbitrary shapes using Bloom filters. However, their scheme failed to address the efficiency issue, where the computational complexity is proportional to the scale of dataset. To improve query efficiency, Wang *et al.* [10] designed a geometric range query scheme that utilizes a dictionary the coordinate distribution range of the data to reduce the computational overhead. The complexity of their scheme is positively related to the size of the dictionary, which is almost the coordinate space if the data distribution range is very wide. Xu *et al.* [14] utilized polynomial fitting technique to fit curve functions to the search curve and compared the function values of the data coordinates to perform geometric range queries. This scheme provides a significant improvement in efficiency, but the misjudgment of range intersection leads to many unnecessary comparison calculations. Wang *et al.* [19] proposed several privacy-preserving Boolean range query schemes [31] and ensured computational efficiency by constructing different index tree structures. However, their scheme cannot protect the privacy of the trapdoor index due to the nature of the applied encryption method SHVE. Tong *et al.* [32] designed a privacy-preserving Boolean range query scheme with temporal access control, but sacrificed computational efficiency for the purpose.

Actually, location privacy protection in location-based services are extensively studied [33], [34]. In [35] and [36], the way to achieve location privacy is to use k -anonymity and dummy users to hide the real location in a cloak area or a group. Although the server is hard to deduce the real identity of the user, the uploaded location information already causes some location privacy leakage. A grid-based hash location protection method on spatial crowdsourcing in [37] has the potential to be extended to a geometric range query scheme, but due to the collision-resistant nature of the hash function, the scheme cannot achieve data irrelevance.

VIII. CONCLUSION

In this article, based on the threat model of the semihonest cloud server, we propose a secure privacy-preserving geographic keyword Boolean range query scheme (EPBRQ) to support searching over encrypted spatial dataset. This scheme not only protects the privacy of the queries and data, but also improves search efficiency. Particularly, we design a recoding algorithm to break the limits of the current work to achieve lower time complexity and use secure Knn computation to protect user data security. In theory, we prove the scheme has the security level of IND-SCAP. And extensive experiments based on real-world dataset demonstrate that the solution can well protect the privacy of data and queries from cloud server threats, and provide better query efficiency than existing works.

For future work, we give two insight research suggestions. First, for more complex cloud server threat model, not a semi-honest but an untrusted could server, more secure searchable encryption techniques are worthy studied to resist the malicious attack. Besides, in this topic, how to balance the privacy protection and search efficiency is a big challenge. Second, the current work on geographic keyword range queries only supports low-frequency small-scale updates of the index structure because of the coupling between tree nodes. That means that an update of one data requires updating multiple nodes, which results in heavy overhead. So, it is a practical and important issue to research the dynamic insertion and deletion to update the index tree with minimal cost.

REFERENCES

- [1] K. Xue, S. Li, J. Hong, Y. Xue, N. Yu, and P. Hong, "Two-cloud secure database for numeric-related sql range queries with privacy preserving," *IEEE Trans. Inf. Forensics Secur.*, vol. 12, no. 7, pp. 1596–1608, Jul. 2017.
- [2] N. Cui, J. Li, X. Yang, B. Wang, M. Reynolds, and Y. Xiang, "When geo-text meets security: Privacy-preserving Boolean spatial keyword queries," in *Proc. IEEE 35th Int. Conf. Data Eng.*, 2019, pp. 1046–1057.
- [3] Q. Wang, M. He, M. Du, S. S. M. Chow, R. W. F. Lai, and Q. Zou, "Searchable encryption over feature-rich data," *IEEE Trans. Dependable Secure Comput.*, vol. 15, no. 3, pp. 496–510, May/Jun. 2018.
- [4] D. X. Song, D. Wagner, and A. Perrig, "Practical techniques for searches on encrypted data," in *Proc. IEEE Symp. Secur. Privacy*, 2000, pp. 44–55.
- [5] W. Zhang, Y. Lin, S. Xiao, J. Wu, and S. Zhou, "Privacy preserving ranked multi-keyword search for multiple data owners in cloud computing," *IEEE Trans. Comput.*, vol. 65, no. 5, pp. 1566–1577, May 2016.
- [6] W. Zhang, Y. Lin, and G. Qi, "Catch you if you misbehave: Ranked keyword search results verification in cloud computing," *IEEE Trans. Cloud Comput.*, vol. 6, no. 1, pp. 74–86, Jan.–Mar. 2018.
- [7] Z. Xia, X. Wang, X. Sun, and Q. Wang, "A secure and dynamic multi-keyword ranked search scheme over encrypted cloud data," *IEEE Trans. Parallel Distrib. Syst.*, vol. 27, no. 2, pp. 340–352, Feb. 2016.
- [8] N. Cao, C. Wang, M. Li, K. Ren, and W. Lou, "Privacy-preserving multi-keyword ranked search over encrypted cloud data," *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 1, pp. 222–233, Jan. 2014.
- [9] J. Shu, X. Jia, K. Yang, and H. Wang, "Privacy-preserving task recommendation services for crowdsourcing," *IEEE Trans. Serv. Comput.*, vol. 14, no. 1, pp. 235–247, Jan./Feb. 2021.
- [10] B. Wang, M. Li, and L. Xiong, "FastGeo: Efficient geometric range queries on encrypted spatial data," *IEEE Trans. Dependable Secure Comput.*, vol. 16, no. 2, pp. 245–258, Mar./Apr. 2019.
- [11] B. Wang, M. Li, H. Wang, and H. Li, "Circular range search on encrypted spatial data," in *Proc. IEEE 35th Int. Conf. Distrib. Comput. Syst.*, 2015, pp. 794–795.
- [12] B. Wang, M. Li, and H. Wang, "Geometric range search on encrypted spatial data," *IEEE Trans. Inf. Forensics Secur.*, vol. 11, no. 4, pp. 704–719, Apr. 2016.

- [13] E. Shen, E. Shi, and B. Waters, "Predicate privacy in encryption systems," in *Theory of Cryptography*, O. Reingold, Ed., Berlin, Germany: Springer, 2009, pp. 457–473.
- [14] G. Xu, H. Li, Y. Dai, K. Yang, and X. Lin, "Enabling efficient and geometric range query with access control over encrypted spatial data," *IEEE Trans. Inf. Forensics Secur.*, vol. 14, no. 4, pp. 870–885, Apr. 2019.
- [15] W. K. Wong, D. W.-I. Cheung, B. Kao, and N. Mamoulis, "Secure KNN computation on encrypted databases," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2009, pp. 139–152.
- [16] Y. Zhou, X. Xie, C. Wang, Y. Gong, and W.-Y. Ma, "Hybrid index structures for location-based web search," in *Proc. 14th ACM Int. Conf. Inf. Knowl. Manage.*, 2005, pp. 155–162. [Online]. Available: <https://doi.org/10.1145/1099554.1099584>
- [17] I. De Felipe, V. Hristidis, and N. Risse, "Keyword search on spatial databases," in *Proc. IEEE 24th Int. Conf. Data Eng.*, 2008, pp. 656–665.
- [18] C. Zhang, Y. Zhang, W. Zhang, and X. Lin, "Inverted linear quadtree: Efficient top k spatial keyword search," *IEEE Trans. Knowl. Data Eng.*, vol. 28, no. 7, pp. 1706–1721, Jul. 2016.
- [19] X. Wang *et al.*, "Search me in the dark: Privacy-preserving Boolean range query over encrypted spatial data," in *Proc. IEEE Conf. Comput. Commun.*, 2020, pp. 2253–2262.
- [20] S. Lai *et al.*, "Result pattern hiding searchable encryption for conjunctive queries," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 745–762.
- [21] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Commun. ACM*, vol. 13, no. 7, pp. 422–426, Jul. 1970.
- [22] J. R. Bitner, G. Ehrlich, and E. M. Reingold, "Efficient generation of the binary reflected gray code and its applications," *Commun. ACM*, vol. 19, no. 9, pp. 517–521, Sep. 1976.
- [23] R. A. Finkel and J. L. Bentley, "Quad trees a data structure for retrieval on composite keys," *Acta Inform.*, vol. 4, no. 1, pp. 1–9, Mar. 1974.
- [24] R. Canetti, S. Halevi, and J. Katz, "A forward-secure public-key encryption scheme," in *Advances in Cryptology – Eurocrypt*. Berlin, Germany: Springer, 2003, pp. 255–271.
- [25] E.-J. Goh, "Secure indexes," *Cryptol. ePrint Arch.*, Tech. Rep. 2003/216, 2003. [Online]. Available: <http://eprint.iacr.org/2003/216/>
- [26] R. Curtmola, J. Garay, S. Kamara, and R. Ostrovsky, "Searchable symmetric encryption: Improved definitions and efficient constructions" in *Proc. 13th ACM Conf. Comput. Commun. Secur.*, 2006, pp. 79–88.
- [27] C. Wang, N. Cao, K. Ren, and W. Lou, "Enabling secure and efficient ranked keyword search over outsourced cloud data," *IEEE Trans. Parallel Distrib. Syst.*, vol. 23, no. 8, pp. 1467–1479, Aug. 2012.
- [28] N. Cao, C. Wang, M. Li, K. Ren, and W. Lou, "Privacy-preserving multi-keyword ranked search over encrypted cloud data," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, 2011, pp. 829–837.
- [29] C. Gentry, "Fully homomorphic encryption using ideal lattices," in *Proc. 41st Annu. ACM Symp. Theory Comput.*, 2009, pp. 169–178.
- [30] H. Zhu, R. Lu, C. Huang, L. Chen, and H. Li, "An efficient privacy-preserving location-based services query scheme in outsourced cloud," *IEEE Trans. Veh. Technol.*, vol. 65, no. 9, pp. 7729–7739, Sep. 2016.
- [31] X. Wang, J. Ma, F. Li, X. Liu, Y. Miao, and R. H. Deng, "Enabling efficient spatial keyword queries on encrypted data with strong security guarantees," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 4909–4923, 2021.
- [32] Q. Tong, X. Li, Y. Miao, X. Liu, J. Weng, and R. Deng, "Privacy-preserving Boolean range query with temporal access control in mobile computing," *IEEE Trans. Knowl. Data Eng.*, to be published, doi: [10.1109/TKDE.2022.3152168](https://doi.org/10.1109/TKDE.2022.3152168).
- [33] Y. Xiao, L. Xiong, S. Zhang, and Y. Cao, "Loclok: Location cloaking with differential privacy via hidden Markov model," *Proc. VLDB Endow.*, vol. 10, no. 12, pp. 1901–1904, Aug. 2017.
- [34] Y. Cao, Y. Xiao, L. Xiong, L. Bai, and M. Yoshikawa, "Protecting spatiotemporal event privacy in continuous location-based services," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 8, pp. 3141–3154, Aug. 2021.
- [35] B. Niu, Q. Li, X. Zhu, G. Cao, and H. Li, "Achieving k-anonymity in privacy-aware location-based services," in *Proc. IEEE Conf. Comput. Commun.*, 2014, pp. 754–762.
- [36] F. Fei, S. Li, H. Dai, C. Hu, W. Dou, and Q. Ni, "A K-anonymity based schema for location privacy preservation," *IEEE Trans. Sustain. Comput.*, vol. 4, no. 2, pp. 156–167, Apr.–Jun. 2019.
- [37] D. Yuan, Q. Li, G. Li, Q. Wang, and K. Ren, "PriRadar: A privacy-preserving framework for spatial crowdsourcing," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 299–314, 2020.



Zhimao Gong received the B.S. degree in software engineering in 2020 from Hunan University, Changsha, China, where he is currently working toward the M.S. degree in computer science and technology with the College of Computer Science and Electronic Engineering.

His research interests include cloud computing security and blockchain.



Junyi Li received the B.S., M.S., and Ph.D. degrees in computer application from Hunan University, Changsha, China, in 1993, 2001, and 2008, respectively.

Since 2005, he has been an Associate Professor with Hunan University. From 2009 to 2010, he was a Visiting Researcher with Lakehead University, Thunder Bay, ON, Canada. His research interests include Big Data analysis, software engineering, privacy protection, and blockchain.



Yaping Lin (Member, IEEE) received the B.S. degree in computer application from Hunan University, Changsha, China, in 1982, the M.S. degree in computer application from the National University of Defense Technology, Changsha, China, in 1985, and the Ph.D. degree in control theory and application from Hunan University, in 2000.

He has been a Professor and Ph.D. Supervisor with Hunan University since 1996. From 2004 to 2005, he was a Visiting Researcher with The University of Texas at Arlington, Arlington, TX, USA. His research

interests include privacy preserving, cloud computing, machine learning, and network security.



Jianhao Wei received the B.S. degree in information and computing science from Shangqiu Normal University, Shangqiu, China, in 2014, the M.S. degree in computer application from Hunan Normal University, Changsha, China, in 2017, and the Ph.D. degree in computer science and technology from the College of Computer Science and Electronic Engineering, Hunan University, Changsha, China, in 2021.

He is currently a Lecturer with the Hunan University of Technology and Business, Changsha, China. His research interests include security and privacy issues in mobile crowdsourcing, cloud computing, and Big Data analysis.



Camara Lancine received the Ph.D. degree in computer application from Hunan University, Changsha, China, in 2015.

He is currently an Associate Professor with the Social Sciences and Management University of Bamako, Bamako, Mali. His research interests include information security and database application testing.